



**SİVAS
UNIVERSITY OF
SCIENCE AND TECHNOLOGY**

**COMPUTER ENGINEERING
MODULE PROJECT
GROUP 12
“AI-Based Maze Game”**

STUDENT’S NAME : Malik ALHASAN
STUDENT’S NUMBER : 123456789

STUDENT’S NAME : -----
STUDENT’S NUMBER : 123456789

STUDENT’S NAME : -----
STUDENT’S NUMBER : 123456789

EXPERIMENT DATE : 24 OCTOBER 2025
DELIVERY DATE : 25 DECEMBER 2025
INSTRUCTOR’S NAME : Assoc. Prof. Dr. Farhan AADIL
INSTRUCTOR’S NAME : Asst. Prof. Dr. Rezan BAKIR

TABLE OF CONTENTS

1	DEFINITION OF THE GAME.....	4
1.1	Game Overview	4
1.2	User Interface and Game Flow	4
1.3	Game Modes and Customization.....	4
1.4	Gameplay Mechanics	5
1.5	Game Management.....	6
1.6	Leaderboard System	7
2	OBJECT-ORIENTED PROGRAMMING	8
2.1	Encapsulation.....	8
2.1.1	Private Fields (Data Hiding):.....	8
2.1.2	Controlled Access (Getters and Setters):.....	8
2.1.3	Data Integrity and Logic Encapsulation:	9
2.1.4	Other Instances of Encapsulation:	9
2.2	Inheritance	10
2.2.1	Extending the Window Class:	10
2.2.2	Overriding Methods (Customizing Behavior):.....	10
2.2.3	Other Instances of Inheritance:.....	11
2.3	Polymorphism & Interfaces.....	11
2.3.1	Custom Interfaces (The Callback Pattern):	11
2.3.2	Anonymous Interface Implementation:	12
2.3.3	Standard Java Interfaces:	13
2.4	Abstraction & Composition.....	13
2.4.1	Composition (Building the Game Manager):	13
2.4.2	Abstraction in AI Logic (Hiding Complexity):	13
2.4.3	Other Instances of Abstraction & Composition:	14
2.5	System Design Diagrams	15
2.5.1	UML Diagram	15
2.5.2	State Diagram	15
2.5.3	Data Flow Diagram.....	16
3	DATA STRUCTURES	18

3.1	Maze Representation: 2D Arrays	18
3.2	AI Pathfinding Data Structures: The Stack	18
3.3	Optimization & State Tracking: Boolean Arrays & HashSets	19
3.4	Dynamic Data Management: ArrayList	20
3.5	Algorithmic Complexity Analysis (Big O)	21
4	CONCLUSION	22
5	PROJECT REPOSITORY & SOURCE CODE	22

1 DEFINITION OF THE GAME

1.1 Game Overview

"Maze Race" is a desktop software application developed using Java and Swing. It simulates a competitive race through a complex labyrinth structure. The application features a dynamic engine that generates a unique, solvable maze for every new session using a recursive backtracking algorithm, ensuring that no two games are identical.

1.2 User Interface and Game Flow

The application features a responsive user interface for intuitive navigation. Upon launching the game, the **Main Menu** serves as the central hub for selecting game modes.



Figure 1: The Main Menu interface allowing users to select between game modes.

1.3 Game Modes and Customization

The system supports two distinct modes of play:

1. **Human vs. AI:** A single-player mode where the user races against an autonomous computer bot.
2. **Human vs. Human:** A local multiplayer mode where two users play on the same screen (Player 1 uses WASD, Player 2 uses Arrow Keys).

Before starting a match, the **Customization Screen** allows players to input their names and select specific avatar colors.

The figure shows two side-by-side setup dialog boxes. The left box is titled 'HUMAN VS HUMAN' and the right box is titled 'HUMAN VS AI'. Both boxes have a subtitle 'Customize Your Players'. In the 'HUMAN VS HUMAN' box, there are two player sections: 'PLAYER 1 (WASD)' and 'PLAYER 2 (ARROW KEYS)'. Each section has a 'Name:' label, a text input field (containing 'Player 1' and 'Player 2' respectively), and a 'Color:' label with eight color swatches. In the 'HUMAN VS AI' box, there is a 'PLAYER 1' section and an 'AI OPPONENT' section. The 'AI OPPONENT' section has a 'Name:' label, a text input field (containing 'AI'), and a 'Color:' label with eight color swatches. Both boxes have a green 'START GAME' button and a red 'CANCEL' button at the bottom.

Figure 2: The Setup Dialog allowing player name input and color selection.

1.4 Gameplay Mechanics

Once the game starts, the screen is divided into two identical panels displaying the same maze.

- **Start:** Player 1 (Blue) and Player 2 (Red) start at the top-left corner.
- **Objective:** Navigate through the white pathways to reach the purple exit portal at the bottom-right corner.
- **Timers:** Real-time timers track exactly how long each competitor takes.

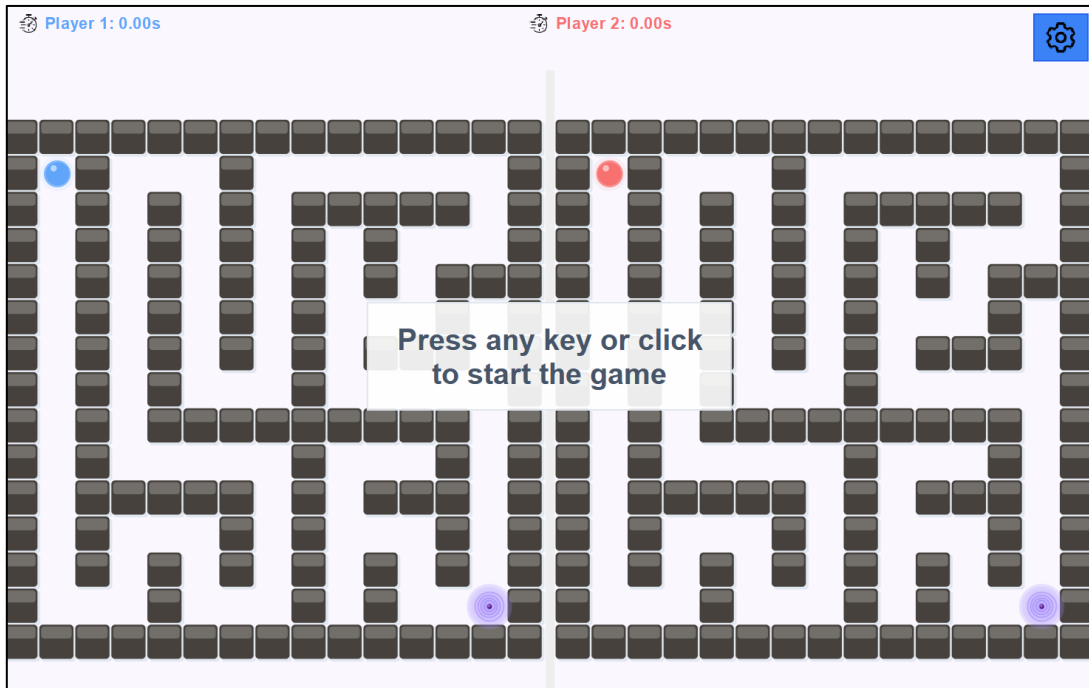


Figure 3: The initial gameplay state waiting for user input.

As the game progresses, the system enforces collision detection with walls. In AI mode, the computer opponent moves autonomously in real-time.

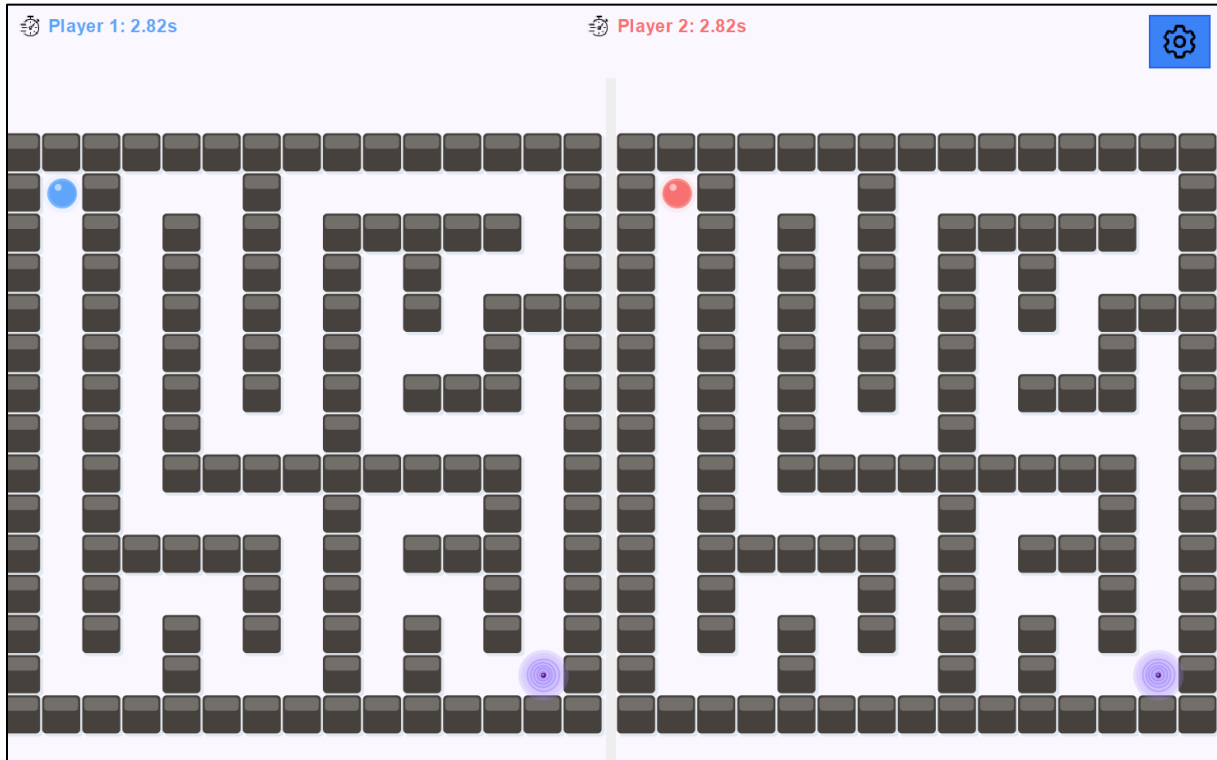


Figure 4: Active gameplay showing both players navigating separate paths.

1.5 Game Management

The game includes a robust management system to ensure a smooth experience.

- **Pause Menu:** Players can pause anytime using the "ESC" key to Resume, Restart, or Exit.

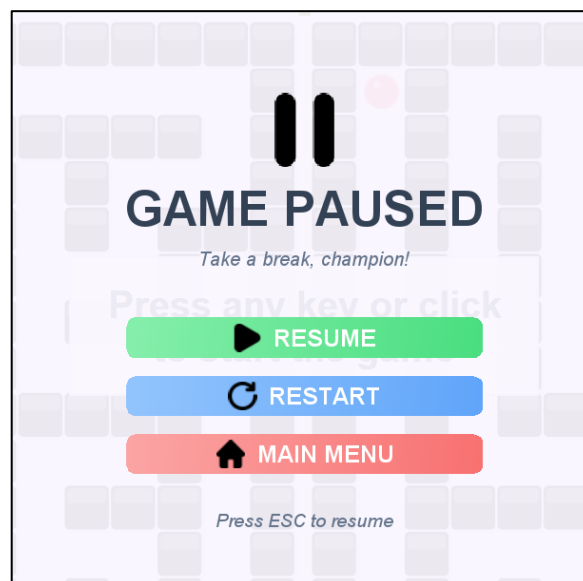


Figure 5: The Pause Menu overlay.

- **Win State:** When a player reaches the portal, the game stops the timers and declares a winner based on the fastest time.

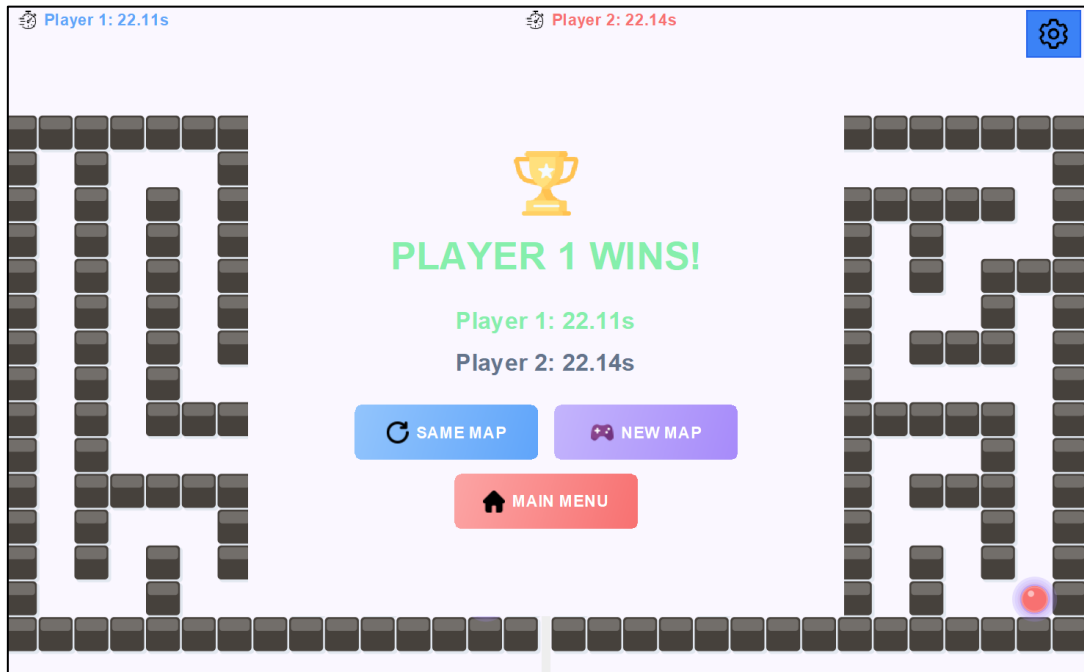


Figure 6: The Game Over screen displaying the winner.

1.6 Leaderboard System

The **Leaderboard System** utilizes local file I/O to save high scores. It tracks the top 10 fastest times and saves the specific maze layout of those games. When a stored map is selected for replay, a selection menu is displayed, allowing the user to choose between "Human vs. AI" or "Human vs. Human" modes for that specific configuration.

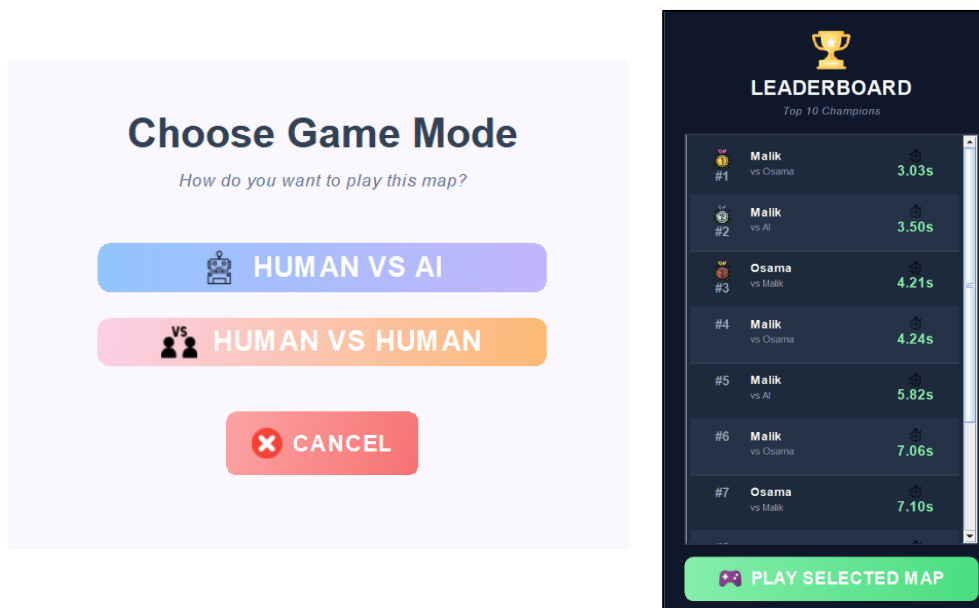


Figure 7: The Leaderboard screen displaying top champions and the Game Mode selection menu that appears when replaying a stored map.

2 OBJECT-ORIENTED PROGRAMMING

2.1 Encapsulation

Concept Overview: Encapsulation is one of the fundamental pillars of Object-Oriented Programming. It involves bundling the data (attributes) and the methods (functions) that operate on the data into a single unit (class). Crucially, it restricts direct access to some of an object's components, which is often referred to as "data hiding."

In this project, Encapsulation is primarily used to protect the integrity of the game state. By keeping critical data `private` and exposing it only through `public` methods, the application ensures that the game's status cannot be corrupted by external classes or inconsistent modifications.

Implementation in the Project: The strongest example of encapsulation in the codebase is the `GameState` class. This class acts as the "source of truth" for the game, managing player positions, game status, and timing.

2.1.1 Private Fields (Data Hiding):

In `GameState.java`, all critical attributes are declared as `private`. This prevents other classes (like the `MazePanel` or `MazeRaceGame`) from directly modifying variables, which could lead to bugs or race conditions.

```
public class GameState {  
    // ===== POSITIONS =====  
    private Point playerPos;      // Player 1 position  
    private Point player2Pos;    // Player 2/AI position  
    private Point exit;          // Exit position  
  
    // ===== STATE FLAGS =====  
    private boolean playerFinished = false; // Has Player 1 finished?  
    private boolean player2Finished = false; // Has Player 2/AI finished?  
    private boolean gameStarted = false;    // Has the game started?  
    // ...  
}
```

By making fields like `playerFinished` or `playerPos` `private`, the class ensures that no external code can accidentally set a player's status to "finished" without the proper checks and timestamp recording occurring.

2.1.2 Controlled Access (Getters and Setters):

Instead of direct access, the `GameState` class provides specific public methods to read and write data. This allows the class to add logic (such as validation or synchronization) whenever data is accessed.

For example, `setPlayerPos` doesn't just update the variable; it encapsulates a thread-safety mechanism (synchronization) to ensure that the position isn't read and written simultaneously

by different threads (the Game Timer thread vs. the UI thread).

```
// From GameState.java
/**
 * Set Player 1's position (thread-safe)
 * @param x New x coordinate
 * @param y New y coordinate
 */
public void setPlayerPos(int x, int y) {
    synchronized(playerLock) {
        playerPos.x = x;
        playerPos.y = y;
    }
}
```

2.1.3 Data Integrity and Logic Encapsulation:

Encapsulation also enables complex logic to be hidden behind simple method calls. The `getPlayerTime()` method calculates the elapsed time based on the current system time, rather than storing a constantly updating "current time" variable. The user of this class doesn't need to know how the time is calculated, only that the method returns the correct value.

```
// From GameState.java
public double getPlayerTime() {
    if (playerFinishTime > 0) {
        // Player finished - return completion time
        return (playerFinishTime - playerStartTime) / 1000.0;
    }
    if (!gameStarted) {
        // Game not started - return 0
        return 0.0;
    }
    // Game ongoing - calculate current elapsed time
    return (System.currentTimeMillis() - playerStartTime) / 1000.0;
}
```

2.1.4 Other Instances of Encapsulation:

While GameState is the primary example, Encapsulation is utilized throughout the project to maintain modularity:

- **MazePanel.java**: Encapsulates the rendering properties. Fields like `maze`, `playerPosition`, and `playerColor` are private. External classes cannot draw directly on the panel; they must update the state via setters (e.g., `setPlayerPosition`) and let the panel handle the `paintComponent` logic internally.
- **Leaderboard.java**: Hides the file I/O operations. The entries list and the save file logic (`SAVE_FILE`) are private. The rest of the application simply calls `addScore()` or `getEntries()` without knowing how the data is serialized to or deserialized from the disk.
- **GameUIManager.java**: Manages the complex UI components (colors, fonts, layout) internally. It exposes only high-level methods like `createMainMenuPanel()` or `showGameEndDialog()`, keeping the specific Swing component configurations hidden from the main game logic.

2.2 Inheritance

Concept Overview: Inheritance is a mechanism in Object-Oriented Programming where a new class (subclass) acquires the properties and behaviors (methods) of an existing class (superclass). This establishes an "Is-A" relationship between the classes.

In this project, inheritance is heavily utilized to build the Graphical User Interface (GUI). Instead of creating UI components from scratch, the game classes extend standard Java Swing classes (`JFrame`, `JPanel`, `JDialog`). This allows the custom classes to inherit complex window management capabilities while adding specific game functionality.

Implementation in the Project: The primary example of inheritance in the application is the `MazeRaceGame` class, which extends `JFrame`.

2.2.1 Extending the Window Class:

By declaring `public class MazeRaceGame extends JFrame`, the `MazeRaceGame` class automatically gains all the attributes of a standard application window. It inherits methods such as `setSize()`, `setLocationRelativeTo()`, and `add()`.

This means `MazeRaceGame` is a `JFrame`, but a specialized one. The code does not need to redefine how to minimize a window, close it, or handle the title bar; it inherits these behaviors directly from the parent class.

```
// From MazeRaceGame.java
public class MazeRaceGame extends JFrame {
    /** Constructor - Initializes the game window and all components */
    public MazeRaceGame() {
        setTitle("Maze Race"); // Inherited method
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Inherited method
        setSize(1200, 800); // Inherited method
        setLocationRelativeTo(null); // Inherited method
        setLayout(new CardLayout()); // Inherited method
        // ...
    }
}
```

2.2.2 Overriding Methods (Customizing Behavior):

Inheritance also allows subclasses to modify the behavior of inherited methods. A clear example of this is in the `MazePanel` class.

`MazePanel` extends `JPanel` but overrides the `paintComponent(Graphics g)` method. While a standard `JPanel` simply paints a background color, `MazePanel` overrides this to draw the custom 3D walls, the player sphere, and the exit portal. The `@Override` annotation ensures that the game's custom rendering logic replaces the default painting logic, while still calling `super.paintComponent(g)` to ensure the panel cleans itself correctly before drawing.

```
// From MazePanel.java
public class MazePanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        // Call parent method first

        // Custom game rendering logic
        if (maze == null) return;
        // ... drawing walls and players ...
    }
}
```

2.2.3 Other Instances of Inheritance:

Inheritance is applied consistently across the project to create specialized UI components:

- `InGameMenu` extends `JDialog`: This class inherits from `JDialog` to create a modal window (a pop-up that blocks other input). It adds custom behaviors like transparency and specific button layouts (Resume, Restart, Main Menu) that a standard `JDialog` does not possess.
- `LeaderboardCellRenderer` extends `DefaultListCellRenderer`: Located inside `GameUIManager`, this class extends the default list renderer to customize how leaderboard entries look. Instead of displaying a simple string, it overrides the rendering method to display medals, icons, and formatted text for each score.
- `GradientButton` (**Anonymous Class**): In `GameUIManager`, buttons are created by extending `JButton` on the fly (via anonymous inner classes) to override `paintComponent` for custom gradient backgrounds.

2.3 Polymorphism & Interfaces

Concept Overview:

- **Polymorphism** (meaning "many forms") allows objects to be treated as instances of their parent class or interface. This enables flexibility, as code can be written to handle a general type rather than a specific one.
- **Interfaces** define a "contract"—a list of methods that a class must implement, without specifying how to implement them. They are crucial for decoupling code, allowing different modules to communicate without needing to know the inner workings of one another.

Implementation in the Project: Implementation in the Project: The project utilizes Polymorphism and Interfaces primarily to handle communication between independent modules and to process user input events.

2.3.1 Custom Interfaces (The Callback Pattern):

To separate the visual elements from the game rules, a custom interface named `UICallback` is defined within the `GameUIManager` class.

- **The Contract:** This interface declares specific methods (e.g., `onAIModeSelected`) that must be executed when a game mode is chosen, but it does not contain the code to actually start the game.
- **The Usage:** The `GameUIManager` class interacts solely with this generic interface. It does not need to know about the `MazeRaceGame` class directly. This ensures that the UI manager focuses only on display logic, while the action of starting a game is delegated to a listener.

```
// From GameUIManager.java
public interface UICallback {
    /** Called when AI mode is selected */
    void onAIModeSelected(String playerName, Color playerColor, Color aiColor);

    /** Called when 2-player human mode is selected */
    void onHumanModeSelected(String player1Name, Color player1Color,
                             String player2Name, Color player2Color);

    /** Called when user wants to play a stored map from leaderboard */
    void onPlayStoredMap(int[][] map, boolean isAIMode,
                        String player1Name, Color player1Color,
                        String player2Name, Color player2Color);
}
```

2.3.2 Anonymous Interface Implementation:

In the main `MazeRaceGame` class, this interface is implemented dynamically using an Anonymous Inner Class.

- **The Implementation:** Instead of creating a separate file for the listener, the interface is implemented "on the fly" when setting up the UI manager. The specific logic for initializing the game state and showing the game panel is written here.
- **The Result:** When a button is clicked in the UI, the generic `onAIModeSelected` method is called. Due to polymorphism, the specific code defined inside `MazeRaceGame` is executed.

```
// From MazeRaceGame.java
private void setupUICallbacks() {
    uiManager.setCallback(new GameUIManager.UICallback() {
        /** Called when user selects AI mode from menu */
        @Override
        public void onAIModeSelected(String playerName, Color playerColor,
                                     Color aiColor) {
            isAIMode = true;
            player1Color = playerColor;
            player2Color = aiColor;
            if (gameState != null) {
                gameState.setPlayer1Name(playerName);
                gameState.setPlayer2Name("AI");
            }
            uiManager.savePlayerInfo(playerName, playerColor, "AI",
                                     aiColor, true);
            showGamePanel(playerName, "AI");
        }
        // ...
    });
}
```

2.3.3 Standard Java Interfaces:

The application also relies on standard Java interfaces to handle user interactions.

- **ActionListener:** Buttons (like "Start Game") require an action listener. By passing a lambda expression (e.g., `e -> startGame()`) to the button, the `ActionListener` interface is implemented polymorphically to define what happens upon a click.
- **KeyListener:** The `MazeRaceGame` class implements the `KeyListener` interface. This allows the main window to listen for keyboard events (like pressing 'W' or 'A') and translate generic key presses into specific player movements.

2.4 Abstraction & Composition

Concept Overview:

- **Abstraction** is the process of hiding complex implementation details and showing only the essential features of an object. It reduces complexity by allowing different parts of the system to interact with high-level methods without needing to understand the underlying logic.
- **Composition** is a design principle where a class is built by combining objects of other classes, establishing a "Has-A" relationship (e.g., a `Car` has an `Engine`). This differs from Inheritance ("Is-A") and is used to assemble complex functionality from smaller, reusable modules.

Implementation in the Project: The project structure relies heavily on Composition to organize the main game class, and Abstraction to simplify the interaction between these components.

2.4.1 Composition (Building the Game Manager):

The central class, `MazeRaceGame`, does not contain all the code for generating mazes, managing high scores, or handling UI layouts. Instead, it utilizes Composition by maintaining instances of specialized classes as private fields.

- **The Structure:** The `MazeRaceGame` class "has" a `Leaderboard`, a `MazeGenerator`, and a `GameUIManager`.
- **The Benefit:** This delegates responsibility. The game logic coordinates these objects, but the specific tasks are handled inside the component classes.

```
// From MazeRaceGame.java
public class MazeRaceGame extends JFrame {
    // Composition: The game is built from these specialized objects
    private GameUIManager uiManager;    // Manages UI creation and updates
    private Leaderboard leaderboard;    // High score management
    private MazeGenerator mazeGenerator; // Maze generation utility
    // ...
}
```

2.4.2 Abstraction in AI Logic (Hiding Complexity):

The `AIPlayer` class provides a clear example of Abstraction. The algorithm used for pathfinding is a complex Depth-First Search (DFS) involving stacks, visited arrays, and dead-end detection.

- **The Interface:** The main game loop simply calls the public method `move(maze, exit)`.
- **The Hidden Detail:** The complex logic of how the next move is calculated—detecting walls, backtracking from dead ends, and randomizing decisions—is completely hidden (abstracted) from the `MazeRaceGame` class. The game loop does not need to know how the AI decides; it only needs the result.

```
// From AIPlayer.java
/**
 * Move the AI player one step towards the exit
 * Uses DFS algorithm with dead-end detection and backtracking
 */
public void move(int[][] maze, Point exit) {
    // Complex logic involving Stacks and HashSets is hidden here
    // ...
}
```

2.4.3 Other Instances of Abstraction & Composition:

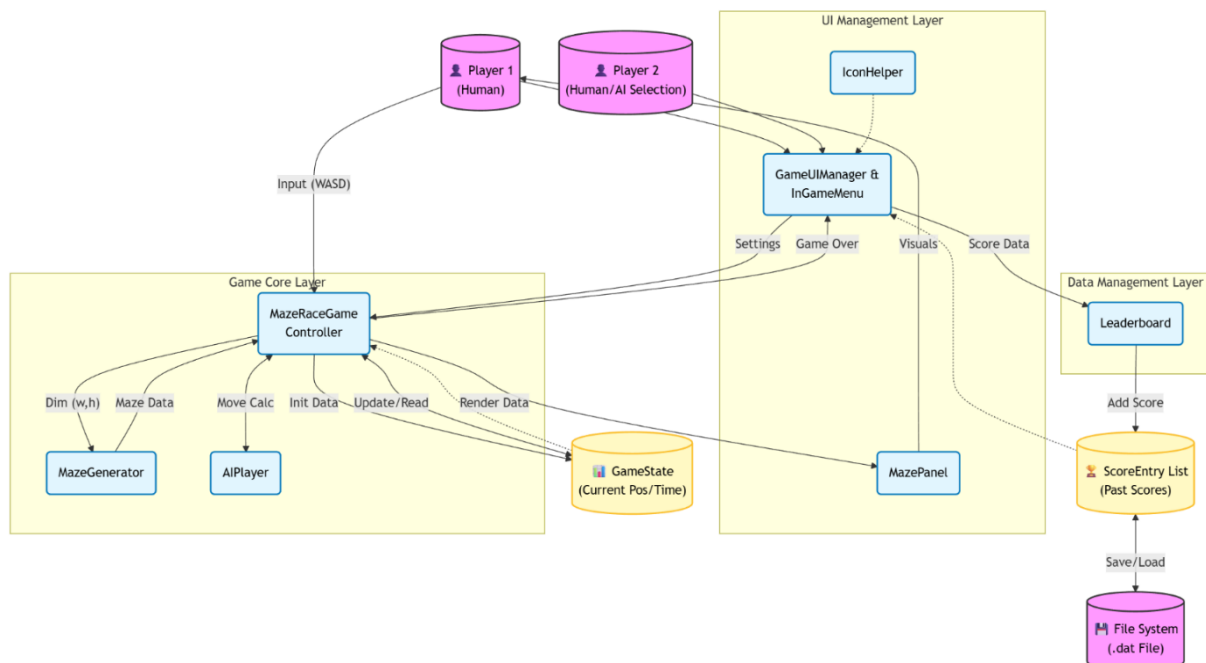
- **Data Persistence (`Leaderboard.java`):** Abstracts file I/O operations. The application calls high-level methods like `addScore()` without handling the low-level details of object serialization or file streams directly.
- **Resource Management (`IconHelper.java`):** Encapsulates the logic for loading and scaling images. UI components retrieve icons via a simple `getIcon()` call, unaware of the underlying URL handling and error protection.
- **Algorithmic Logic (`MazeGenerator.java`):** Hides the recursive backtracking generation logic. The game requests a maze via `generate()` and receives a result without managing the generation state.

2.5 System Design Diagrams

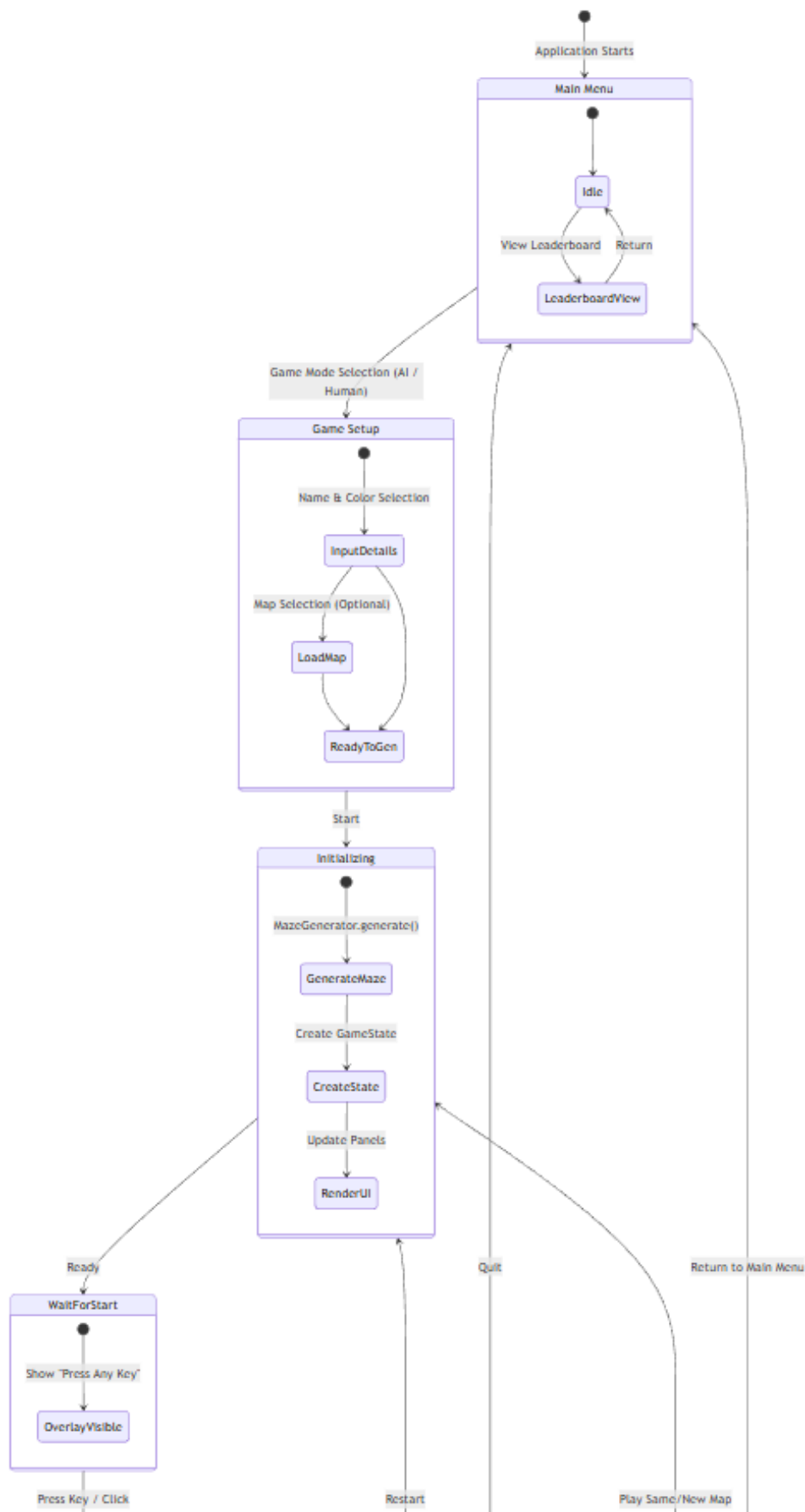
2.5.1 UML Diagram

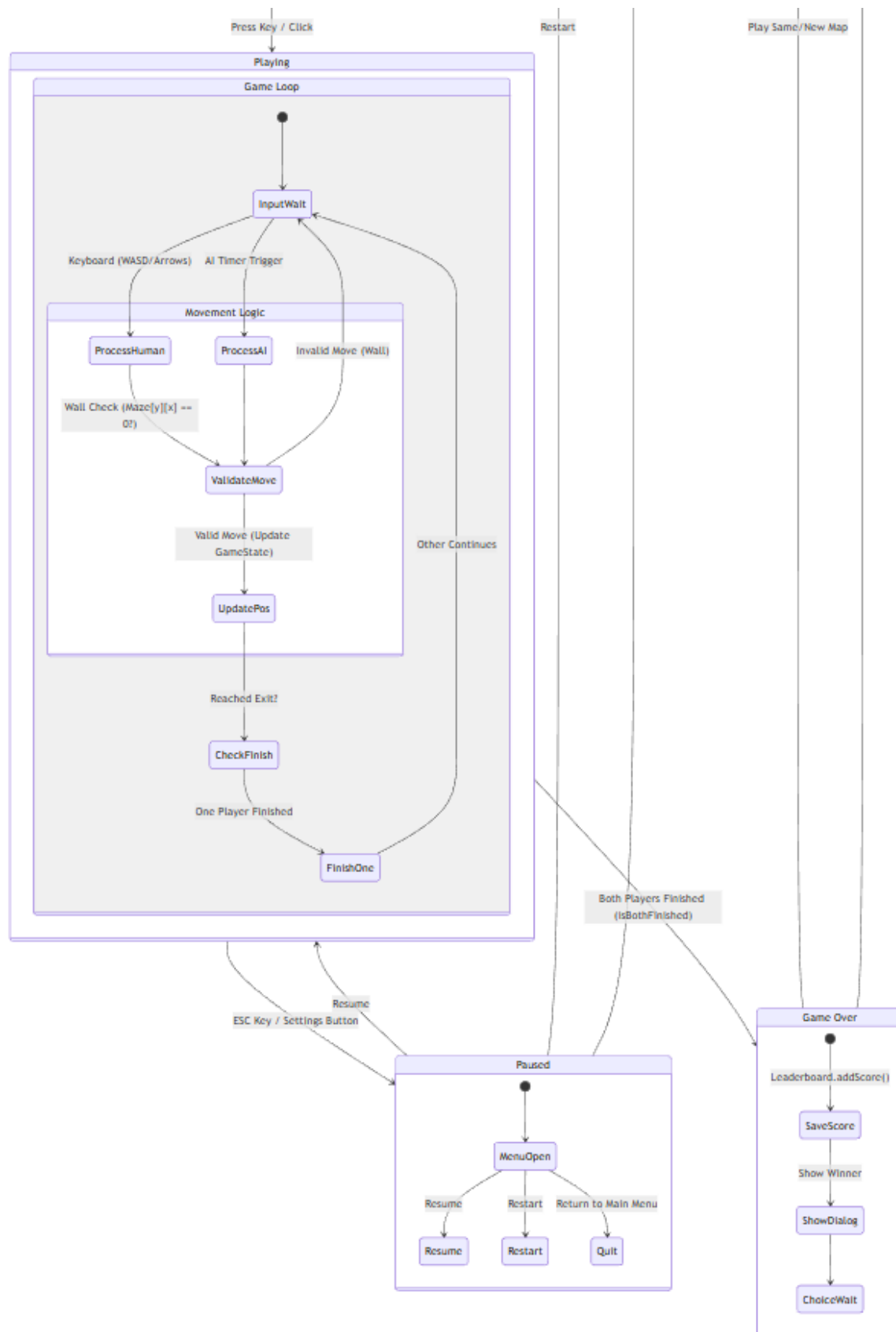


2.5.2 State Diagram



2.5.3 Data Flow Diagram





3 DATA STRUCTURES

3.1 Maze Representation: 2D Arrays

Concept Overview:

The fundamental data structure used to represent the game world (the maze) is the 2-Dimensional Array (Matrix). In computer science, a 2D array is essentially a "list of lists" that organizes data into a grid format consisting of rows and columns. This structure provides a direct mathematical mapping between the data in memory and the visual components on the user's screen.

Implementation in the Project:

The maze layout is stored as a 2D integer array (`int[][] maze`).

- **Dimensions:** The outer array represents the rows (Y-axis), and the inner arrays represent the columns (X-axis).
- **Cell States:** Integers are used to define the state of each specific block in the grid. For example:
 - 1 represents a **Wall** (Impassable object).
 - 0 represents a **Path** (Walkable area).
 - 9 represents the **Exit Portal**.

Justification & Analysis:

Why was a 2D Array chosen over other structures like a Graph (Adjacency List) or a Linked List?

1) Direct Access Speed (Time Complexity $O(1)$):

The most critical requirement for a real-time game is collision detection. The game loop runs approximately 60 times per second and must constantly check: "Is the player walking into a wall?"

a) **2D Array:** We can access any specific cell immediately using its coordinates: `maze[row][col]`. This is an $O(1)$ (Constant Time) operation.

b) **Linked List:** Checking a coordinate would require traversing the list from the start, resulting in $O(N)$ complexity, which is too slow for real-time physics.

2) Memory Mapping:

Since the game is visually rendered as a grid of square tiles, a matrix is the most logical representation. The x, y pixel coordinates of the player on the screen map linearly to the `[row][col]` indices of the array.

3) Implicit Connectivity:

Unlike a generic Graph structure where we must explicitly store "Edges" (connections) between nodes, a 2D Array has implicit connectivity. We know that `maze[x][y]` is connected to `maze[x+1][y]` simply because of their indices. We do not need to waste memory creating "Edge" objects for every walkable path.

3.2 AI Pathfinding Data Structures: The Stack

Concept Overview:

The primary data structure driving the AI's decision-making logic is the **Stack**. A Stack is a linear data structure that follows the **LIFO (Last-In, First-Out)** principle. This means the last element added to the collection is the first one to be removed. It can be visualized like a stack of plates: you can only add or remove a plate from the top.

Implementation in the AI:

In this project, the Stack is utilized to implement the **Depth-First Search (DFS)** algorithm.

- **Push:** When the AI moves to a new cell, that cell's coordinates are "Pushed" onto the stack. This represents the current path taken.
- **Pop:** When the AI reaches a dead end (a wall or a cell with no unvisited neighbors), it "Pops" the current cell off the stack. This action effectively moves the AI backward to the previous intersection.
- **Top:** The AI always looks at the "Top" of the stack to know its current location and decide the next move.

Justification:

Why Stack (DFS) instead of Queue (BFS)? The choice of a Stack over a Queue was a deliberate design decision based on the behavior we wanted for the AI.

1) Backtracking Capability (The Primary Reason):

- a) **Stack (LIFO):** The Stack is naturally designed for **backtracking**. When solving a maze, if a player takes a wrong turn, they must retrace their steps to the last junction. A Stack handles this automatically: popping elements reverses the path history one step at a time.
- b) **Queue (FIFO):** A Queue follows the **First-In, First-Out** principle and is used for Breadth-First Search (BFS). BFS explores the maze layer-by-layer. It does not naturally support the "forward-then-backward" movement of a single agent exploring a maze.

2) Path Simulation vs. Shortest Path:

- a) **Queue (BFS):** BFS is mathematically guaranteed to find the *shortest* path. However, in a "Maze Race" context, we wanted the AI to behave like a human competitor.
- b) **Stack (DFS):** DFS plunges deep into a path until it hits a wall. This results in the AI making "mistakes" and having to turn back, which simulates realistic human behavior and creates a fairer, more exciting race for the human player. A BFS-based AI would simply flow perfectly to the exit, often beating the human too easily.

3) Memory Efficiency:

- a) In a maze with long, narrow corridors, a Stack (DFS) typically consumes less memory. It only needs to store the nodes on the *current single path*.
- b) A Queue (BFS) would need to store every node at the current "frontier" of the search. In a large maze, this "frontier" can grow very large, consuming significantly more memory just to decide the next move.

3.3 Optimization & State Tracking: Boolean Arrays & HashSets

Concept Overview:

To prevent the AI from getting stuck in infinite loops or wasting time processing blocked paths, the system must efficiently track the state of the maze. We utilize a hybrid approach combining **2D Boolean Arrays** and **HashSets** to optimize performance.

Implementation in the AI:

1. **Visited Array (boolean[][]):** The `AIPlayer` class uses a 2D boolean array named `visited`. This maps directly to the maze grid, where `true` means a cell has been explored.
2. **Dead Ends (HashSet):** A `HashSet<Point>` named `deadEnds` is used to store coordinates that have been identified as having no exit.

Justification:

- **Why boolean[][] for Visited?** For a dense grid where the AI might visit most cells, a boolean array is faster than any other structure. Checking `visited[y][x]` is a direct

memory lookup ($O(1)$). It avoids the overhead of creating thousands of object wrappers just to say "I was here."

- **Why HashSet for Dead Ends?** Dead ends are sparse (there are fewer of them than total cells). A HashSet is ideal here because it allows us to instantly check `deadEnds.contains(point)` in $O(1)$ time. This prevents the AI from re-entering a corridor it has already proven is useless, significantly speeding up the solving process.

3.4 Dynamic Data Management: ArrayList

Concept Overview:

In Java, a standard array (e.g., `String[]`) has a fixed size that must be declared at the moment of creation. Once created, it cannot grow or shrink. To overcome this limitation, we utilize the **ArrayList** class. An **ArrayList** is a "resizable array" implementation of the List interface. It internally manages a standard array but automatically creates a larger one and copies data over whenever the capacity is reached.

Implementation in the Leaderboard:

The project features a **Leaderboard System** that saves and loads high scores from a local text file.

- When the game launches, the system reads the save file line-by-line.
- Since the file could theoretically contain 0 scores (empty), 5 scores, or 100 scores, we cannot know the exact size needed beforehand.
- We use an `ArrayList<ScoreEntry>` to store these records as they are parsed from the file.
- After loading, the list is sorted, and if it exceeds 10 entries, the extra elements are removed to keep only the "Top 10".

Justification:

Why ArrayList instead of Standard Array ([])?

While standard arrays are slightly faster in raw performance, they are unsuitable for file management for two key reasons:

1) Unknown Data Volume (Flexibility):

- a) **Standard Array:** If we used `ScoreEntry[] scores = new ScoreEntry[10];`, we would face immediate issues. What if the file has fewer than 10 lines? We would have null values complicating the logic. What if we decide later to track the top 50 scores? We would have to rewrite the initialization code.
- b) **ArrayList:** The **ArrayList** handles the volume dynamically. We can simply call `.add()` without worrying about the underlying capacity. It decouples the *storage logic* from the *game rules* (e.g., the rule "Top 10" is applied logically after loading, not physically restricted by the container size).

2) Built-in Manipulation Methods:

- a) **Sorting:** The **ArrayList** integrates seamlessly with `Collections.sort()`. To sort a custom object (like a high score) using a standard array requires more verbose code.
- b) **Removal:** When a new high score beats the 10th place, we need to insert the new score and remove the old one. In a standard array, removing an item requires writing a loop to manually shift every subsequent element one slot to the left. The **ArrayList** provides the `.remove()` method, which handles this memory shifting internally and efficiently.

3.5 Algorithmic Complexity Analysis (Big O)

Concept Overview:

Algorithmic complexity quantifies how the performance of an algorithm degrades as the input size grows. We use Big O Notation to describe the "Worst-Case Scenario." For our Maze Game, the "Input Size" (N) is effectively the total number of cells in the grid ($Rows \times Columns$).

1) Maze Generation (Recursive Backtracking)

- a) **Time Complexity:** $O(N)$ (Linear Time) The recursive generator visits every cell in the grid to determine if it should be carved into a path or left as a wall. Crucially, it marks cells as "visited" immediately. This ensures that the algorithm never processes the same cell twice. Therefore, the time taken is directly proportional to the total number of cells.
- b) **Space Complexity:** $O(N)$ Because the algorithm is recursive, it utilizes the system's "Call Stack" to remember where it came from. In the worst-case scenario (a long, winding path that snakes through the entire grid without branching), the recursion depth could equal the total number of cells, requiring linear memory.

2) AI Pathfinding (Depth-First Search)

- a) **Time Complexity:** $O(V + E) \approx O(N)$ In Graph Theory, the maze is a graph where cells are Vertices (V) and open paths are Edges (E).
 - i) The DFS algorithm explores each reachable vertex and traverses each edge at most twice (once forward, once backtracking).
 - ii) Since $V = N$ (total cells) and each cell has a maximum of 4 neighbors (constant number of edges), the complexity simplifies to $O(N)$.
 - iii) **Justification:** This "Linear Complexity" is highly efficient for real-time gaming. It ensures that the AI can calculate moves instantly without causing the game to "lag" or freeze, even if we increase the map size.
- b) **Space Complexity:** $O(N)$ The Stack data structure stores the current path being explored. In the worst case (if the solution path is extremely long), the stack might need to store coordinates for a large portion of the maze. Additionally, the **visited boolean array** requires fixed memory proportional to the grid size, and the **deadEnds HashSet** stores invalid paths. Thus, memory usage grows linearly with the map size.

3) Conclusion on Efficiency

- a) We deliberately avoided algorithms with Quadratic Complexity ($O(N^2)$) or Exponential Complexity ($O(2^N)$). Such algorithms would check "every cell against every other cell," which would make the game unplayable on larger grids. By using $O(1)$ access for our data structures (Arrays and HashSets) and **Linear Time** $O(N)$ algorithms for logic, we ensured the software remains responsive and lightweight.

4 CONCLUSION

The "Maze Race Game" project successfully implemented a real-time, AI-driven simulation by strictly applying Object-Oriented Programming (OOP) principles and efficient Data Structures. By utilizing **Encapsulation** for state management and **Inheritance** for the UI, we created a modular and maintainable system. Furthermore, the strategic use of **Stacks** for DFS pathfinding and **HashSets** for optimization ensured that both maze generation and the AI solver operate with **Linear Time Complexity ($O(N)$)**, resulting in a responsive and high-performance application.

5 PROJECT REPOSITORY & SOURCE CODE

The complete source code, including commit logs, and asset resources, is available in the public GitHub repository below.

- **Project Link:** <https://github.com/MalikFikret/MazeRaceGameProject>